📘 Linguaggi e Metodologie di Programmazione

Modulo II: Programmazione Dichiarativa e Funzionale

> **Corso:** Università di Roma "Tor Vergata" - ART (Artificial Intelligence Research)
> **Docente:** Fabio Massimo Zanzotto
> **Materiale:** Slide del corso + dispense programmazione funzionale (Lorenzo Ferrone)

🗂️ Indice dei Contenuti

Parte I - Paradigma Dichiarativo (Prolog)

1. 🧠 Lezione 0 - Introduzione al Paradigma Dichiarativo
2. 📐 Lezione 1 - Basi del Linguaggio Prolog
3. ⚡ Lezione 2 - Esecuzione dei Programmi, Liste e Operatori Aritmetici
4. 🏗️ Lezione 3 - Operatori e Strutture Dati
5. 🔍 Lezione 4 - Controllare lo Spazio di Ricerca
6. 🧩 Lezione 5 - Controllare Tipi Dati, Decomporre Fatti, Controllare il Database
7. 📥 Lezione 6 - Recuperare Tutte le Risposte e Gestione I/O
8. 🔄 Lezione 7 - Ordinamento e Operazioni su Dati Strutturati
9. 🌲 Lezione 8 - Similarità tra Alberi e Programmazione Dinamica
10. ⚖️ Lezione 9 - Alberi Bilanciati
11. 🎯 Lezione 10 - Strategie di Problem Solving
12. 💬 Lezione 11 - Cenni di Natural Language Processing in Prolog
13. 📖 Lezione 12 - Cenni di Logica
14. 🤖 Lezione 13 - Cenni di Machine Learning in Prolog

Parte II - Paradigma Funzionale (Python)

15. 🐍 Parte II - Python e Programmazione Funzionale

PARTE I - PARADIGMA DICHIARATIVO (PROLOG)

🧠 Lezione 0 - Introduzione al Paradigma Dichiarativo

Concetti Fondamentali

Il Problema della Computazione

Nei linguaggi di programmazione tradizionali (imperativi/procedurali), risolviamo i problemi attraverso **procedure** che sono sequenze di ordini. Queste sono raccolte in funzioni, procedure e/o metodi.

> **Tesi di Church-Turing:** Tutto ciò che è computabile è risolvibile attraverso macchine basate su procedure.

Programmazione Imperativa vs Dichiarativa

Paradigma	Come pensa	Esempio
Imperativo	"Come" fare le cose (sequenza di istruzioni)	Java, C, Python
Dichiarativo	"Cosa" vogliamo ottenere (regole e vincoli)	Prolog, SQL
Funzionale	Funzioni matematiche, immutabilità	Haskell, Lisp, Python (parziale)

Esempi di Problemi Affrontati

🔤 Esempio 1: Costruire un Cruciverba

```
% Parole disponibili:
% dog, four, baker, prolog, run, lost, forum, vanish, top, mess,
% green, wonder, five, unit, super, yellow

% In Prolog dichiariamo le parole e i vincoli di incrocio,
% poi il sistema trova TUTTE le soluzioni automaticamente!
```

> 💡 **Vantaggio Prolog:** Non scriviamo l'algoritmo di ricerca, descriviamo solo il problema (vincoli) e Prolog trova le soluzioni.

🗺️ Esempio 2: Colorare una Mappa (Graph Colouring)

Problema classico: usando un numero limitato di colori, trovare una colorazione tale che stati adiacenti non abbiano lo stesso colore.

```
% In Prolog:
color(rosso). color(blu). color(verde). color(giallo).

% Vincolo: stati adiacenti devono avere colori diversi
valid_coloring :-
    italia(C1), francia(C2), C1 \= C2,
    italia(C1), svizzera(C3), C1 \= C3,
    % ... altri vincoli
    italia(C1), austria(C4), C1 \= C4.
```

📚 Materiale di Riferimento

- **Libro consigliato:** Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Addison Wesley (4th edition)
- **Programma del corso:**
  1. Programmazione ad Oggetti (Java) - già visto nel I modulo
  2. **Programmazione Dichiarativa (Prolog)** - questo modulo
  3. **Programmazione Funzionale (Python)** - seconda parte

📐 Lezione 1 - Basi del Linguaggio Prolog

Elementi Fondamentali

1. Termini

Atomi (costanti simboliche):

- Lettera minuscola all'inizio: `andrea`, `bob`
- Tra apici singoli: `'Corso di Prolog'`
- Numeri con caratteri: `c1p8`

Numeri:

- `12345`, `3.14`

Variabili:

- Lettera MAIUSCOLA o `_` all'inizio: `Tizio`, `_andrea`, `_`
- `_` è la variabile anonima (non ci interessa il valore)

Termini composti:

- `somma(1, 2, X)`, `person(bob, smith, 30)`
- Funtore + argomenti

2. Predicati

```
% Notazione: funtore/arietà
parent/2        % predicato parent con 2 argomenti
grandparent/2   % predicato grandparent con 2 argomenti
```

3. Clausole: Fatti e Regole

Fatti (fatti senza condizioni):

```
% Sintassi: predicato(arg1, arg2, ...).
parent(anne, bill).
parent(anne, charlie).
parent(bill, donnie).
friend(luke, daisy).
```

Regole (clausole con testa e corpo):

```
% Sintassi: Head :- Body.
% Leggere: "Head è vera SE Body è vero"

grandparent(X, Y) :-
    parent(X, Z),      % X è genitore di Z
    parent(Z, Y).      % Z è genitore di Y
    % Quindi X è nonno di Y
```

Operatori nel corpo:

- `,` = AND logico
- `;` = OR logico
- `.` = fine della regola

4. Programma Logico

Un programma Prolog è un **insieme di regole e fatti**. Risponde alle query con:

- `true` / `false`
- Valori assegnati alle variabili

🖥️ Esempi Pratici - Famiglia

```
% ===== DATABASE =====
parent(anne, bill).
parent(anne, charlie).
parent(bill, donnie).
```

```
% ===== REGOLE =====
grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).

sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.  % X diverso da Y
```

Query di base:

```
?- parent(anne, bill).
true.

?- parent(anne, X).
X = bill ;      % premi ';' per la prossima soluzione
X = charlie ;
false.           % non ci sono più soluzioni

?- parent(X, Y).
X = anne, Y = bill ;
X = anne, Y = charlie ;
X = bill, Y = donnie ;
false.

?- grandparent(X, donnie).
X = anne.        % anne -> bill -> donnie

?- sibling(bill, charlie).
true.
```

🔑 Come Funziona Prolog (prima intuizione)

1. **Cerca** nel database dall'alto verso il basso
2. **Unifica** variabili con valori (pattern matching)
3. **Backtracking**: se un percorso fallisce, torna indietro e prova alternative
4. **Ricorsione**: le regole possono chiamare se stesse

> ⚠️ **Importante:** L'ordine dei fatti e delle regole conta in Prolog!

Lezione 2 - Esecuzione dei Programmi, Liste e Operatori Aritmetici

Algoritmo di Risoluzione (Esempio Passo-Passo)

```
% ===== DATABASE =====
edge(a, b).
edge(b, c).
edge(a, e).
edge(c, d).
edge(d, e).
edge(f, e).

% Percorso diretto
path(X, Y) :- edge(X, Y).

% Percorso indiretto (ricorsivo)
```

```
path(X, Y) :-
    path(X, Z),
    path(Z, Y).
```

Traccia esecuzione: `?- path(a, d).`

```
path(a, d)
├── path(a, Z), path(Z, d)
    ├── edge(a, Z), path(Z, d)    % unifica Z = b
    │   ├── edge(a, b) ✓
    │   └── path(b, d)
    │       ├── edge(b, d) ✗      % non esiste!
    │       └── path(b, Z2), path(Z2, d)
    │           ├── edge(b, Z2)   % Z2 = c
    │           │   ├── edge(b, c) ✓
    │           │   └── path(c, d)
    │           │       ├── edge(c, d) ✓
    │           │       └── SUCCESSO!
```

Lettura dei Programmi Prolog

Lettura Dichiarativa (logica)

```
% "Per ogni X, Y, Z: X è nonno di Y SE X è genitore di Z E Z è genitore di Y"
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

Lettura Procedurale (esecuzione)

```
% "Per soddisfare grandparent(X,Y), prima soddisfa parent(X,Z),
%  poi soddisfa parent(Z,Y)"
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

Liste in Prolog

Sintassi delle Liste

```
% Lista vuota
[]

% Lista con elementi
[1, 2, 3]
[anna, bob, charlie]

% Testa e Coda (Head e Tail)
[H | T]
% H = primo elemento
% T = lista dei restanti elementi

?- [H|T] = [1, 2, 3, 4].
H = 1, T = [2, 3, 4].
```

Esempi di Unificazione con Liste

```
?- [X, Y, Z] = [1, 2, 3].
X = 1, Y = 2, Z = 3.

?- [X | T] = [1, 2, 3].
```

```
X = 1, T = [2, 3].

?- [X, Y | T] = [1, 2, 3, 4].
X = 1, Y = 2, T = [3, 4].

?- [_, _, X | _] = [1, 2, 3, 4, 5].
X = 3.          % terzo elemento
```

Operazioni Fondamentali sulle Liste

1. Member (appartenenza):

```
% Versione base
member(X, [X | _]).           % X è in testa
member(X, [_ | T]) :-
    member(X, T).             % X è nella coda

?- member(2, [1, 2, 3]).
true.

?- member(X, [1, 2, 3]).
X = 1 ; X = 2 ; X = 3 ; false.
```

2. Concatenazione (Append):

```
append([], L, L).
append([H | T], L, [H | R]) :-
    append(T, L, R).

?- append([1, 2], [3, 4], X).
X = [1, 2, 3, 4].

?- append(X, Y, [1, 2, 3]).  % Genera tutte le suddivisioni!
X = [], Y = [1, 2, 3] ;
X = [1], Y = [2, 3] ;
X = [1, 2], Y = [3] ;
X = [1, 2, 3], Y = [].
```

3. Lunghezza:

```
length([], 0).
length([_ | T], N) :-
    length(T, N1),
    N is N1 + 1.

?- length([1, 2, 3, 4], N).
N = 4.
```

Operatori Aritmetici

```
% 'is' evalua l'espressione e assegna
?- X is 3 + 4.
X = 7.

?- X is 10 // 3.    % divisione intera
X = 3.

?- X is 10 mod 3.   % resto
```

```prolog
X = 1.

% Operatori di confronto
% =:=  (uguale valore)
% =\= (diverso valore)
% <, >, =<, >=

?- 2 + 3 =:= 5.
true.
```

> ⚠️ **ATTENZIONE:** `=` unifica, `is` evalua!
>
> ```prolog
> ?- X = 3 + 4.      % X = 3+4 (termine composto!)
> ?- X is 3 + 4.     % X = 7 (valore numerico)
> ```

🏗️ Lezione 3 - Operatori e Strutture Dati

Strutture Dati Complesse

In Prolog le strutture dati sono create tramite **termini composti** (funtori con argomenti).

Esempio: Famiglia Complessa

```prolog
% ===== STRUTTURA DATI: FAMIGLIA =====
% family(Padre, Madre, ListaFigli)
% person(Nome, Cognome, DataNascita, Salario)

family(
    person(bob, smith, date(7, may, 1968), 30000),
    person(ann, smith, date(18, july, 1970), 32000),
    [
        person(dave, smith, date(1, june, 1984), 0),
        person(edna, smith, date(25, may, 1990), 0)
    ]
).

% ===== REGOLE DI ACCESSO =====

% Chi è il marito?
husband(X) :- family(X, _, _).

% Chi è la moglie?
wife(X) :- family(_, X, _).

% Chi è figlio?
child(X) :-
    family(_, _, Children),
    member(X, Children).

% Esiste nella famiglia
exists(X) :- husband(X) ; wife(X) ; child(X).

% Estrai salario
salary(person(_, _, _, S), S).

% Estrai data di nascita
dateOfBirth(person(_, _, Date, _), Date).
```

```
% ===== QUERY =====
?- exists(person(Nome, smith, _, _)).
Nome = bob ;
Nome = ann ;
Nome = dave ;
Nome = edna.

?- salary(person(bob, smith, date(7, may, 1968), S), Salario).
Salario = 30000.
```

Esempio: Automa Non Deterministico (NFA)

```
% ===== NFA: RICONOSCIMENTO STRINGHE =====
% final(State) - stati finali
% trans(State, Char, NextState) - transizioni
% empty(State, NextState) - epsilon-transizioni

final(s3).

trans(s0, a, s1).
trans(s1, a, s1).
trans(s1, b, s2).
trans(s2, b, s3).

empty(s2, s0).

% ===== ACCETTAZIONE =====
accepts(State, []) :- final(State).
accepts(State, [X|Rest]) :-
    trans(State, X, Next),
    accepts(Next, Rest).
accepts(State, String) :-
    empty(State, Next),
    accepts(Next, String).

?- accepts(s0, [a, a, b, b]).
true.    % stringa accettata!
```

Esempio: Problema delle 8 Regine ♛

```
% ===== 8 REGINE =====
% Posizionare 8 regine su una scacchiera senza che si minaccino

% Genera liste di permutazioni
template([1/_, 2/_, 3/_, 4/_, 5/_, 6/_, 7/_, 8/_]).

% Soluzione: trova configurazione valida
solution([]) :- !.
solution([X/Y | Others]) :-
    solution(Others),
    member(Y, [1,2,3,4,5,6,7,8]),
    noattack(X/Y, Others).

% Nessuna regina minaccia un'altra
noattack(_, []).
noattack(X/Y, [X1/Y1 | Others]) :-
    Y =\= Y1,                  % colonne diverse
    Y1-Y =\= X1-X,             % non stessa diagonale
    Y1-Y =\= X-X1,
    noattack(X/Y, Others).
```

```
% Trova tutte le soluzioni!
?- template(T), solution(T).
```

🔍 Lezione 4 - Controllare lo Spazio di Ricerca

Backtracking e il Taglio (!)

Il Problema

```
% Esempio problematico: minimo
del_min(X, [X]).
del_min(Min, [H | T]) :-
    del_min(MinT, T),
    H > MinT,          % se H > min della coda
    Min is MinT.       % allora min è MinT
del_min(Min, [H | T]) :-
    del_min(MinT, T),
    H =< MinT,         % se H <= min della coda
    Min is H.          % allora min è H

% PROBLEMA: se la prima regola fallisce,
% Prolog prova la seconda anche quando non dovrebbe!
```

La Soluzione: il Cut (!)

Il **cut** `!` blocca il backtracking. Una volta passato il cut, Prolog non può più tornare indietro.

```
% Versione corretta con CUT
del_min(X, [X]).
del_min(Min, [H | T]) :-
    del_min(MinT, T),
    H > MinT,
    !,                 % ← CUT: se siamo qui, non tornare indietro
    Min is MinT.
del_min(Min, [H | T]) :-
    del_min(MinT, T),
    H =< MinT,
    !,                 % ← CUT
    Min is H.
```

Tipi di Cut

Cut	Nome	Uso
Green Cut	Cut "verde"	Migliora efficienza senza cambiare risultati
Red Cut	Cut "rosso"	Cambia il comportamento del programma (essenziale per correttezza)

Esempio: not_member con Cut

```
% Verifica che X NON sia nella lista L
not_member(_, []) :- !.              % caso base
t_not_member(X, [Y | T]) :-
    X \= Y,
    !,
```

```
    not_member(X, T).

?- not_member(4, [1, 2, 3]).
true.

?- not_member(2, [1, 2, 3]).
false.
```

Esempio: Massimo tra due numeri

```
max(X, Y, X) :- X >= Y, !.    % se X >= Y, X è il max (stop!)
max(X, Y, Y) :- X < Y.         % altrimenti Y è il max

% SENZA cut (versione problematica):
max2(X, Y, X) :- X >= Y.
max2(X, Y, Y) :- X < Y.
% Se fai backtracking dopo max2(5, 3, X), potresti
% ottenere anche X = 3 come "seconda soluzione"!

% CON cut (versione corretta):
max2(X, Y, X) :- X >= Y, !.
max2(X, Y, Y).
```

!FAIL: Fallimento Esplicito

```
% Vogliamo escludere elementi
% not/1: nega un goal

not(Goal) :- Goal, !, fail.    % se Goal ha successo, fallisci
not(Goal).                      % altrimenti ha successo

% Uso:
not_member(X, L) :- member(X, L), !, fail.
not_member(_, _).
```

🧩 Lezione 5 - Controllare Tipi Dati, Decomporre Fatti, Controllare il Database

Predicati di Controllo dei Tipi

```
% ===== PREDICATI DI TIPO =====

% var(X) - X è una variabile non istanziata
?- var(X).
true.

?- X = 5, var(X).
false.          % X è istanziata!

% nonvar(X) - X è istanziata
?- nonvar(X).
false.
?- X = 5, nonvar(X).
true.

% atom(X) - X è un atomo
?- atom(hello).
true.
```

```
% number(X) - X è un numero
?- number(42).
true.

% atomic(X) - X è atomo o numero
?- atomic(hello).   % true
?- atomic(42).      % true

% compound(X) - X è un termine composto
?- compound(f(a, b)).
true.

% ground(X) - X non contiene variabili
?- ground(f(a, b)). % true
?- ground(f(X, b)). % false
```

Decomporre Fatti e Termini (=.. univ)

L'operatore `=..` converte un termine in lista e viceversa:

```
?- f(a, b, c) =.. List.
List = [f, a, b, c].

?- Term =.. [parent, ann, bill].
Term = parent(ann, bill).

% ===== APPLICAZIONE: Contare elementi in un termine =====
size(Term, 1) :- atomic(Term), !.
size(Term, N) :-
    Term =.. [_ | Args],
    size_list(Args, NArgs),
    N is NArgs + 1.

size_list([], 0).
size_list([H | T], N) :-
    size(H, NH),
    size_list(T, NT),
    N is NH + NT.

?- size(f(a, b(c, d), e), N).
N = 6.
```

Gestione del Database Dinamico

```
% ===== PREDICATI DINAMICI =====
% I predicati dichiarati :- dynamic possono essere
% aggiunti/rimossi a runtime

:- dynamic parent/2.    % dichiara parent/2 come dinamico

% assertz(Fatto) - aggiunge in fondo
?- assertz(parent(tom, bob)).
true.

% asserta(Fatto) - aggiunge in testa
?- asserta(parent(zoe, ann)).
true.

% retract(Fatto) - rimuove una clausola
```

```
?- retract(parent(tom, bob)).
true.

% retractall(Predicato) - rimuove tutte le clausole
?- retractall(parent(_, _)).
true.

% listing(Predicato) - mostra tutte le clausole
?- listing(parent).
```

Esempio Pratico: Accumulatore di Fatti

```
:- dynamic visited/1.

mark_visited(X) :-
    not(visited(X)),      % se non già visitato
    assertz(visited(X)).   % marca come visitato

reset :- retractall(visited(_)).

% Uso:
?- mark_visited(a).
?- mark_visited(b).
?- visited(X).
X = a ; X = b.
```

📥 Lezione 6 - Recuperare Tutte le Risposte e Gestione I/O

Findall, Bagof, Setof

Questi predicati raccolgono TUTTE le soluzioni in una lista.

findall/3

```
% findall(Template, Goal, ResultList)
% Raccoglie TUTTE le istanze di Template che soddisfano Goal

?- findall(X, parent(anne, X), Figli).
Figli = [bill, charlie].

?- findall(Cognome, child(person(_, Cognome, _, _)), Cognomi).
% Raccoglie tutti i cognomi dei figli
```

bagof/3

```
% bagof(Template, Goal, ResultList)
% Come findall ma raggruppa per variabili libere

% Se ci sono variabili libere in Goal, bagof le usa per raggruppare
?- bagof(X, parent(Y, X), FigliDiY).
FigliDiY = [bill, charlie], Y = anne ;
FigliDiY = [donnie], Y = bill.
```

setof/3

```
% setof(Template, Goal, ResultList)
% Come bagof ma ORDINA e RIMUOVE DUPLICATI

?- setof(X, (parent(anne, X); parent(bill, X)), Tutti).
Tutti = [bill, charlie, donnie].    % ordinato!
```

Predicato	Duplicati	Ordinato	Raggruppamento
`findall`	Mantiene	No	Nessuno
`bagof`	Mantiene	No	Per variabili libere
`setof`	Rimuove	Sì	Per variabili libere

Input/Output in Prolog

Output: write/1, nl/0

```
?- write('Hello, World!'), nl.
Hello, World!
true.

?- X = 42, write('Valore: '), write(X), nl.
Valore: 42
```

Input: read/1

```
?- read(X).          % attende input dall'utente
|: 42.
X = 42.

?- read(X).
|: ciao.
X = ciao.            % atomo

?- read(X).
|: 'Ciao mondo'.
X = 'Ciao mondo'.    % stringa

?- read(X).
|: [1, 2, 3].
X = [1, 2, 3].       % lista
```

File I/O

```
% ===== SCRITTURA SU FILE =====
write_data :-
    open('output.txt', write, Stream),
    write(Stream, 'Dati:'),
    nl(Stream),
    write(Stream, [1, 2, 3]),
    close(Stream).

% ===== LETTURA DA FILE =====
read_data :-
    open('input.txt', read, Stream),
    read(Stream, Term),
    write(Term),
    close(Stream).
```

```
% ===== LETTURA LINEA PER LINEA =====
read_lines(Stream) :-
    read_line_to_codes(Stream, Line),
    ( Line \= end_of_file ->
        atom_codes(Atom, Line),
        write(Atom), nl,
        read_lines(Stream)
    ; true ).
```

Esempio Completo: Lettura e Scrittura

```
% Salva database su file
save_db(Filename) :-
    open(Filename, write, S),
    forall(parent(X, Y),
           (writeq(S, parent(X, Y)), write(S, '.'), nl(S))),
    close(S).

% Carica database da file
load_db(Filename) :-
    open(Filename, read, S),
    repeat,
    read(S, Term),
    ( Term = end_of_file -> close(S), !
    ; assertz(Term), fail ).
```

🔄 Lezione 7 - Ordinamento e Operazioni su Dati Strutturati

Algoritmi di Ordinamento in Prolog

Bubble Sort

```
% ===== BUBBLE SORT =====
% Scambia elementi adiacenti fino a ordinare

bubble_sort(List, Sorted) :-
    swap_pass(List, List1), !,   % se c'è stato uno scambio
    bubble_sort(List1, Sorted).
bubble_sort(Sorted, Sorted).     % altrimenti già ordinato

swap_pass([X, Y | Rest], [Y, X | Rest]) :-
    X > Y, !.                    % scambia se fuori ordine
swap_pass([X | Rest], [X | Rest1]) :-
    swap_pass(Rest, Rest1).

?- bubble_sort([3, 1, 4, 1, 5], S).
S = [1, 1, 3, 4, 5].
```

Insertion Sort

```
% ===== INSERTION SORT =====
% Inserisce ogni elemento nella posizione corretta

insertion_sort([], []).
insertion_sort([H | T], Sorted) :-
    insertion_sort(T, SortedT),
```

```
    insert(H, SortedT, Sorted).

insert(X, [], [X]).
insert(X, [H | T], [X, H | T]) :- X =< H, !.
insert(X, [H | T], [H | T1]) :-
    insert(X, T, T1).

?- insertion_sort([3, 1, 4, 1, 5], S).
S = [1, 1, 3, 4, 5].
```

Quick Sort

```
% ===== QUICK SORT =====
% Divide (pivot) e conquista

quicksort([], []).
quicksort([H | T], Sorted) :-
    partition(H, T, Less, Greater),
    quicksort(Less, SortedLess),
    quicksort(Greater, SortedGreater),
    append(SortedLess, [H | SortedGreater], Sorted).

partition(_, [], [], []).
partition(P, [H | T], [H | Less], Greater) :-
    H =< P,
    partition(P, T, Less, Greater).
partition(P, [H | T], Less, [H | Greater]) :-
    H > P,
    partition(P, T, Less, Greater).

?- quicksort([3, 1, 4, 1, 5, 9, 2, 6], S).
S = [1, 1, 2, 3, 4, 5, 6, 9].
```

Merge Sort

```
% ===== MERGE SORT =====
% Divide a metà, ordina ricorsivamente, merge

mergesort([], []) :- !.
mergesort([X], [X]) :- !.
mergesort(List, Sorted) :-
    split(List, L1, L2),
    mergesort(L1, S1),
    mergesort(L2, S2),
    merge(S1, S2, Sorted).

split(List, L1, L2) :-
    length(List, Len),
    Half is Len // 2,
    length(L1, Half),
    append(L1, L2, List).

merge([], L, L) :- !.
merge(L, [], L) :- !.
merge([H1 | T1], [H2 | T2], [H1 | T]) :-
    H1 =< H2, !,
    merge(T1, [H2 | T2], T).
merge([H1 | T1], [H2 | T2], [H2 | T]) :-
    merge([H1 | T1], T2, T).
```

```
?- mergesort([3, 1, 4, 1, 5, 9, 2, 6], S).
S = [1, 1, 2, 3, 4, 5, 6, 9].
```

Operazioni su Alberi

Albero Binario di Ricerca (BST)

```
% ===== BST: RICERCA =====
bst_member(X, node(X, _, _)).
bst_member(X, node(Y, Left, _)) :-
    X < Y,
    bst_member(X, Left).
bst_member(X, node(Y, _, Right)) :-
    X > Y,
    bst_member(X, Right).

% ===== BST: INSERIMENTO =====
bst_insert(X, nil, node(X, nil, nil)).
bst_insert(X, node(Y, L, R), node(Y, L1, R)) :-
    X < Y,
    bst_insert(X, L, L1).
bst_insert(X, node(Y, L, R), node(Y, L, R1)) :-
    X > Y,
    bst_insert(X, R, R1).

% ===== BST: TRAVERSALS =====
inorder(nil, []).
inorder(node(X, L, R), List) :-
    inorder(L, LL),
    inorder(R, LR),
    append(LL, [X | LR], List).

preorder(nil, []).
preorder(node(X, L, R), List) :-
    preorder(L, LL),
    preorder(R, LR),
    append([X | LL], LR, List).

postorder(nil, []).
postorder(node(X, L, R), List) :-
    postorder(L, LL),
    postorder(R, LR),
    append(LL, LR, Sub),
    append(Sub, [X], List).
```

🌲 Lezione 8 - Similarità tra Alberi e Programmazione Dinamica

Distanza di Edit tra Alberi

La **distanza di edit** misura quanto sono diversi due alberi (numero minimo di operazioni per trasformarne uno nell'altro).

```
% ===== OPERAZIONI SULL'ALBERO =====
% - Inserimento di un nodo
% - Cancellazione di un nodo
% - Sostituzione (rilabeling) di un nodo

% ===== DISTANZA DI TAI (1989) =====
% Definizione ricorsiva:
```

```
% Distanza tra foreste
tai_dist([], [], 0).
tai_dist([T | F1], [], D) :-
    size(T, S),
    tai_dist(F1, [], D1),
    D is D1 + S.           % cancella tutti i nodi di T
tai_dist([], [T | F2], D) :-
    size(T, S),
    tai_dist([], F2, D1),
    D is D1 + S.           % inserisci tutti i nodi di T

% Distanza tra alberi
tai_dist(node(L1, F1), node(L2, F2), D) :-
    L1 == L2, !,           % stessa etichetta
    tai_dist(F1, F2, D).
tai_dist(node(_, F1), node(_, F2), D) :-
    tai_dist(F1, F2, D1),
    D is D1 + 1.           % costo sostituzione etichetta

% Distanza minima (considera tutte le operazioni)
tai_dist(T1, T2, D) :-
    findall(Del, (subtree(T1, Sub), tai_del(Sub, T1, T2, Del)), Dels),
    findall(Ins, (subtree(T2, Sub), tai_ins(Sub, T1, T2, Ins)), Inss),
    findall(Rep, tai_rep(T1, T2, Rep), Reps),
    append([Dels, Inss, Reps], All),
    min_list(All, D).
```

Programmazione Dinamica in Prolog

La **programmazione dinamica** è una tecnica che:

1. Divide il problema in sottoproblemi
2. Memoizza (memorizza) le soluzioni dei sottoproblemi
3. Riutilizza le soluzioni già calcolate

```
% ===== FIBONACCI CON MEMOIZZAZIONE =====
:- dynamic fib_memo/2.

fib(N, F) :- fib_memo(N, F), !.    % già calcolato?
fib(0, 0) :- !.
fib(1, 1) :- !.
fib(N, F) :-
    N1 is N - 1,
    N2 is N - 2,
    fib(N1, F1),
    fib(N2, F2),
    F is F1 + F2,
    assertz(fib_memo(N, F)).        % memoizza!

% Senza memoizzazione: O(2^n)
% Con memoizzazione: O(n)
```

Esempio: Knapsack (Zaino) in Prolog

```
% ===== PROBLEMA DELLO ZAINO =====
% Oggetti: peso, valore
item(1, 2, 3).   % item(ID, Peso, Valore)
item(2, 3, 4).
item(3, 4, 5).
```

```
item(4, 5, 6).

% knapsack(Capacità, ValoreMassimo)
knapsack(Cap, Val) :-
    findall((P, V), item(_, P, V), Items),
    knapsack_dp(Items, Cap, Val).

knapsack_dp([], _, 0).
knapsack_dp([(P, V) | Rest], Cap, Val) :-
    P =< Cap,
    !,
    Cap1 is Cap - P,
    knapsack_dp(Rest, Cap1, Val1),
    knapsack_dp(Rest, Cap, Val2),
    Val is max(V + Val1, Val2).
knapsack_dp([_ | Rest], Cap, Val) :-
    knapsack_dp(Rest, Cap, Val).
```

⚖️ Lezione 9 - Alberi Bilanciati

Alberi AVL (Adelson-Velsky e Landis)

Un albero AVL è un BST in cui per ogni nodo, la differenza di altezza tra i sottoalberi sinistro e destro è al massimo 1.

```
% ===== ALBERO AVL =====
% Struttura: node(Valore, Altezza, Sinistro, Destro)

% Altezza di un albero
height(nil, 0).
height(node(_, H, _, _), H).

% Bilanciamento: fattore di bilanciamento
balance(node(_, _, L, R), Bal) :-
    height(L, HL),
    height(R, HR),
    Bal is HL - HR.

% ===== ROTAZIONI =====

% Rotazione destra (LL case)
rotate_right(node(Y, _, node(X, _, A, B), C),
             node(X, _, A, node(Y, _, B, C))).

% Rotazione sinistra (RR case)
rotate_left(node(X, _, A, node(Y, _, B, C)),
            node(Y, _, node(X, _, A, B), C)).

% Rotazione doppia sinistra-destra (LR case)
rotate_lr(node(Z, _, node(X, _, A, node(Y, _, B, C)), D),
          node(Y, _, node(X, _, A, B), node(Z, _, C, D))).

% Rotazione doppia destra-sinistra (RL case)
rotate_rl(node(X, _, A, node(Z, _, node(Y, _, B, C), D)),
          node(Y, _, node(X, _, A, B), node(Z, _, C, D))).

% ===== BILANCIAMENTO =====
balance_tree(T, T1) :-
    balance(T, Bal),
    ( Bal > 1 ->
```

```
        ( T = node(_, _, L, _), balance(L, LBal), LBal >= 0 ->
            rotate_right(T, T1)
        ; rotate_lr(T, T1) )
    ; Bal < -1 ->
        ( T = node(_, _, _, R), balance(R, RBal), RBal =< 0 ->
            rotate_left(T, T1)
        ; rotate_rl(T, T1) )
    ; T1 = T
    ).

% Aggiorna altezza
update_height(node(V, _, L, R), node(V, H, L, R)) :-
    height(L, HL),
    height(R, HR),
    H is max(HL, HR) + 1.

% Inserimento AVL
avl_insert(X, nil, node(X, 1, nil, nil)) :- !.
avl_insert(X, node(V, H, L, R), T) :-
    X < V,
    !,
    avl_insert(X, L, L1),
    update_height(node(V, H, L1, R), T1),
    balance_tree(T1, T).
avl_insert(X, node(V, H, L, R), T) :-
    X > V,
    avl_insert(X, R, R1),
    update_height(node(V, H, L, R1), T1),
    balance_tree(T1, T).
```

Alberi 2-3

```
% ===== ALBERO 2-3 =====
% Nodo 2: node2(Valore, Sinistro, Destro)
% Nodo 3: node3(Val1, Val2, Sinistro, Medio, Destro)

% Ricerca in albero 2-3
tt_search(V, node2(V, _, _)).
tt_search(V, node3(V, _, _, _, _)).
tt_search(V, node3(V2, _, _, _, _)) :- V == V2.
tt_search(V, node2(K, L, _)) :- V < K, tt_search(V, L).
tt_search(V, node2(K, _, R)) :- V > K, tt_search(V, R).
tt_search(V, node3(K1, _, L, _, _)) :- V < K1, tt_search(V, L).
tt_search(V, node3(K1, K2, _, M, _)) :- V > K1, V < K2, tt_search(V, M).
tt_search(V, node3(_, K2, _, _, R)) :- V > K2, tt_search(V, R).
```

🎯 Lezione 10 - Strategie di Problem Solving

Problem Solving in IA con Prolog

Generate and Test

```
% ===== STRATEGIA: GENERATE AND TEST =====
% 1. Genera tutte le possibili soluzioni
% 2. Testa quali soddisfano i vincoli

% Esempio: Trova numeri primi in un range
prime_in_range(From, To, Primes) :-
```

```
    findall(N, (between(From, To, N), is_prime(N)), Primes).

is_prime(2) :- !.
is_prime(N) :-
    N > 2,
    N mod 2 =\= 0,
    Max is floor(sqrt(N)),
    not(has_divisor(N, 3, Max)).

has_divisor(N, D, Max) :-
    D =< Max,
    N mod D =:= 0, !.
has_divisor(N, D, Max) :-
    D2 is D + 2,
    has_divisor(N, D2, Max).
```

Backtracking Ordinato

```
% ===== PERMUTAZIONI =====
% Genera tutte le permutazioni di una lista

permutation([], []).
permutation([H | T], Perm) :-
    permutation(T, PermT),
    insert(H, PermT, Perm).

insert(X, L, [X | L]).
insert(X, [H | T], [H | T1]) :-
    insert(X, T, T1).

?- permutation([1, 2, 3], P).
P = [1, 2, 3] ;
P = [1, 3, 2] ;
P = [2, 1, 3] ;
P = [2, 3, 1] ;
P = [3, 1, 2] ;
P = [3, 2, 1].
```

Constraint Satisfaction Problems (CSP)

```
% ===== SUDOKU SOLVER =====
% Rappresentazione: lista di 81 celle (riga per riga)

sudoku(Puzzle, Solution) :-
    Solution = Puzzle,
    % Vincoli: tutti i numeri 1-9 in ogni riga, colonna, blocco
    rows(Solution, Rows),
    cols(Solution, Cols),
    blocks(Solution, Blocks),
    valid(Rows),
    valid(Cols),
    valid(Blocks).

valid([]).
valid([Group | Groups]) :-
    permutation([1,2,3,4,5,6,7,8,9], Group),
    valid(Groups).

% Estrazione righe, colonne, blocchi
rows([], []).
```

```
rows(Puzzle, [Row | Rows]) :-
    append(Row, Rest, Puzzle),
    length(Row, 9),
    rows(Rest, Rows).

cols(Rows, Cols) :- transpose(Rows, Cols).

blocks(Rows, Blocks) :-
    blocks_rows(Rows, [], Blocks).

% (implementazione dettagliata omessa per brevità)
```

Euristiche di Ricerca

```
% ===== BEST-FIRST SEARCH =====
% Usa un'euristica per guidare la ricerca

best_first(Start, Goal, Path) :-
    best_first([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

best_first([[Goal | Path] | _], Goal, [Goal | Path]).
best_first([Path | Paths], Goal, Solution) :-
    expand(Path, NewPaths),
    append(Paths, NewPaths, AllPaths),
    sort_paths(AllPaths, SortedPaths),  % ordina per euristica
    best_first(SortedPaths, Goal, Solution).

expand([Node | Path], NewPaths) :-
    findall([NewNode, Node | Path],
            (move(Node, NewNode), not(member(NewNode, [Node | Path]))),
            NewPaths).

% Euristica: distanza stimata dal goal
heuristic(Node, Goal, H) :-
    % es: distanza euclidea, manhattan, ecc.
    manhattan_distance(Node, Goal, H).
```

💬 Lezione 11 - Cenni di Natural Language Processing in Prolog

Parsing con Prolog: Definite Clause Grammars (DCG)

Le **DCG** sono un'estensione di Prolog per il parsing di linguaggi naturali.

```
% ===== GRAMMATICA DCG =====
% Frase -> Sintagma_Nominale + Sintagma_Verbale

frase --> sintagma_nominale, sintagma_verbale.

sintagma_nominale --> articolo, nome.
sintagma_nominale --> nome_proprio.

sintagma_verbale --> verbo.
sintagma_verbale --> verbo, sintagma_nominale.

articolo --> [il].
articolo --> [la].
articolo --> [lo].
```

```
nome --> [gatto].
nome --> [cane].
nome --> [palla].

nome_proprio --> [maria].
nome_proprio --> [giovanni].

verbo --> [corre].
verbo --> [mangia].
verbo --> [insegue].

% ===== USO =====
?- phrase(frase, [il, gatto, corre]).
true.

?- phrase(frase, [maria, mangia, la, palla]).
true.

?- phrase(frase, X).        % Genera tutte le frasi!
X = [il, gatto, corre] ;
X = [il, gatto, mangia] ;
X = [il, gatto, mangia, il, gatto] ;
...

% ===== DCG CON SEMANTICA =====
% Aggiungiamo significato alle frasi

frase(Meaning) -->
    sintagma_nominale(Soggetto),
    sintagma_verbale(Soggetto, Meaning).

sintagma_nominale(X) --> articolo, nome(X).
sintagma_nominale(X) --> nome_proprio(X).

sintagma_verbale(S, agire(S)) --> verbo(Intrans).
sintagma_verbale(S, agire_oggetto(S, O)) --> verbo(Trans), sintagma_nominale(O).

articolo --> [il].
articolo --> [la].

nome(gatto) --> [gatto].
nome(palla) --> [palla].

verbo(intrans) --> [corre].
verbo(trans) --> [mangia].
verbo(trans) --> [insegue].

?- phrase(frase(M), [il, gatto, mangia, la, palla]).
M = agire_oggetto(gatto, palla).
```

Features e Unificazione Linguistica

```
% ===== CONCORDANZA =====
% Maschile/Femminile, Singolare/Plurale

frase --> sn(Gen, Num), sv(Gen, Num).

sn(Gen, Num) --> det(Gen, Num), n(Gen, Num).
sn(_, Num) --> np(Num).
```

```
sv(Gen, Num) --> v(Num).

% Determinanti
det(masc, sing) --> [il].
det(fem, sing) --> [la].
det(masc, plur) --> [i].
det(fem, plur) --> [le].

% Nomi
n(masc, sing) --> [gatto].
n(fem, sing) --> [gatta].
n(masc, plur) --> [gatti].
n(fem, plur) --> [gatte].

% Verbi
v(sing) --> [corre].
v(plur) --> [corrono].

?- phrase(frase, [il, gatto, corre]).      % ✓ OK
?- phrase(frase, [la, gatta, corre]).      % ✓ OK
?- phrase(frase, [il, gatto, corrono]).    % ✗ Errore! (non concordano)
```

Parsing Bottom-Up e Top-Down

```
% ===== PARSING TOP-DOWN (ricorsivo discendente) =====
parse_top_down(Categoria, Tokens) :-
    rule(Categoria, RHS),
    match(RHS, Tokens, Remaining),
    Remaining = [].

rule(frase, [sn, sv]).
rule(sn, [det, nome]).
rule(sv, [verbo]).

match([], Tokens, Tokens).
match([Cat | Cats], Tokens, Remaining) :-
    ( lexicon(Cat, Tokens, Rest) ->
        match(Cats, Rest, Remaining)
    ; parse_top_down(Cat, Tokens, Rest),
      match(Cats, Rest, Remaining) ).

lexicon(det, [il | T], T).
lexicon(nome, [gatto | T], T).
lexicon(verbo, [corre | T], T).
```

📖 Lezione 12 - Cenni di Logica

Logica dei Predicati (Logica del Primo Ordine - FOL)

Sintassi

```
Termini:
  - Variabili: x, y, z
  - Costanti: a, b, c
  - Funzioni: f(t1, ..., tn)

Formule:
  - Predicati: P(t1, ..., tn)
```

```
  - Connettivi: ¬ (not), ∧ (and), ∨ (or), → (implica), ↔ (sse)
  - Quantificatori: ∀ (per ogni), ∃ (esiste)
```

Esempi di Formule Logiche

```
"Tutti gli uomini sono mortali"
∀x (Uomo(x) → Mortale(x))

"Socrate è un uomo"
Uomo(socrate)

"Quindi Socrate è mortale" (Modus Ponens)
Mortale(socrate)

"Esiste un numero pari"
∃x Pari(x)

"Ogni numero ha un successore"
∀x ∃y Succ(x, y)
```

Semantica

Interpretazione

Un'**interpretazione** I consiste in:

- Dominio D (insieme non vuoto)
- Assegnazione di costanti → elementi di D
- Assegnazione di funzioni → funzioni su D
- Assegnazione di predicati → relazioni su D

Modello

Un **modello** di una formula F è un'interpretazione che rende F vera.

Deduzione Naturale

Regole di Inferenza

```
┌─────────────────────────────────────┐
│  MODUS PONENS                       │
│  Da A → B e A, inferisce B          │
│                                     │
│  A → B    A                         │
│  ──────────                         │
│      B                              │
└─────────────────────────────────────┘

┌─────────────────────────────────────┐
│  AND INTRODUCTION                   │
│  Da A e B, inferisce A ∧ B          │
│                                     │
│    A      B                         │
│  ──────────                         │
│    A ∧ B                            │
└─────────────────────────────────────┘

┌─────────────────────────────────────┐
│  UNIVERSAL INSTANTIATION            │
│  Da ∀x P(x), inferisce P(a)         │
```

```
┌                                          ┐
│  ∀x P(x)                                 │
│  ──────────                              │
│   P(a)                                   │
└                                          ┘

┌──────────────────────────────────────────┐
│  EXISTENTIAL GENERALIZATION              │
│  Da P(a), inferisce ∃x P(x)              │
│                                          │
│   P(a)                                   │
│  ──────────                              │
│  ∃x P(x)                                 │
└──────────────────────────────────────────┘
```

Risoluzione

```
┌──────────────────────────────────────────┐
│  REGOLA DI RISOLUZIONE                   │
│                                          │
│  A ∨ B    ¬B ∨ C                         │
│  ──────────────────                      │
│      A ∨ C                               │
│                                          │
│  (B e ¬B si cancellano)                  │
└──────────────────────────────────────────┘
```

Forma Clausale (CNF)

Passi per convertire in forma clausale:

1. Eliminare implicazioni: A → B diventa ¬A ∨ B
2. Spingere ¬ verso l'interno (De Morgan)
3. Standardizzare variabili
4. Skolemizzare (eliminare ∃)
5. Convertire in forma prenessa
6. Convertire in CNF (congiunzione di disgiunzioni)
7. Estrarre clausole

Esempio:

```
∀x (Uomo(x) → Mortale(x)) ∧ Uomo(socrate) → Mortale(socrate)

Passo 1: Eliminare →
¬(∀x (¬Uomo(x) ∨ Mortale(x)) ∧ Uomo(socrate)) ∨ Mortale(socrate)

Passo 2: Spingere ¬
∃x (Uomo(x) ∧ ¬Mortale(x)) ∨ ¬Uomo(socrate) ∨ Mortale(socrate)

Passo 3: Skolemizzare
(Uomo(a) ∧ ¬Mortale(a)) ∨ ¬Uomo(socrate) ∨ Mortale(socrate)

Clausole:
1. {Uomo(a), ¬Uomo(socrate), Mortale(socrate)}
2. {¬Mortale(a), ¬Uomo(socrate), Mortale(socrate)}
```

Unificazione

L'**unificazione** trova una sostituzione che rende due termini identici.

```
┌──────────────────────────────────────┐
│  ALGORITMO DI UNIFICAZIONE (Robinson)  │
│                                      │
│  1. Se t1 = t2, successo             │
│  2. Se t1 è variabile, sostituisci   │
│  3. Se t2 è variabile, sostituisci   │
│  4. Se entrambi composti, unifica    │
│     ricorsivamente argomenti         │
│  5. Altrimenti, fallimento           │
└──────────────────────────────────────┘

Esempi:
  f(X, b)  unifica con  f(a, Y)    → {X/a, Y/b}
  f(X, X)  unifica con  f(a, b)    → fallimento (X non può essere a e b)
  f(g(X))  unifica con  f(Y)       → {Y/g(X)}
  X        unifica con  g(X)       → fallimento (occur check)
```

🤖 Lezione 13 - Cenni di Machine Learning in Prolog

Alberi di Decisione in Prolog

```
% ===== ALBERO DI DECISIONE: PLAY TENNIS =====
% Attributi: Outlook, Temperature, Humidity, Wind

% Esempi di training
tennis(sunny, hot, high, weak, no).
tennis(sunny, hot, high, strong, no).
tennis(overcast, hot, high, weak, yes).
tennis(rain, mild, high, weak, yes).
tennis(rain, cool, normal, weak, yes).
tennis(rain, cool, normal, strong, no).
tennis(overcast, cool, normal, strong, yes).
tennis(sunny, mild, high, weak, no).
tennis(sunny, cool, normal, weak, yes).
tennis(rain, mild, normal, weak, yes).
tennis(sunny, mild, normal, strong, yes).
tennis(overcast, mild, high, strong, yes).
tennis(overcast, hot, normal, weak, yes).
tennis(rain, mild, high, strong, no).

% Predizione: chiedi all'albero
predict(Outlook, Temp, Humidity, Wind, Play) :-
    % Regole apprese dall'albero
    ( Outlook = overcast -> Play = yes
    ; Outlook = sunny ->
        ( Humidity = high -> Play = no
        ; Humidity = normal -> Play = yes )
    ; Outlook = rain ->
        ( Wind = strong -> Play = no
        ; Wind = weak -> Play = yes )
    ).

?- predict(sunny, hot, normal, weak, Play).
Play = yes.
```

Regole di Associazione (Apriori)

```
% ===== REGOLE DI ASSOCIAZIONE =====
% Trova pattern frequenti nei dati

% Transazioni: lista di item acquistati
transaction([pane, latte, burro]).
transaction([pane, burro]).
transaction([latte, uova]).
transaction([pane, latte, burro, uova]).
transaction([latte, uova]).

% Supporto di un itemset
support(Itemset, Support) :-
    findall(T, transaction(T), Transactions),
    count_containing(Itemset, Transactions, Count),
    length(Transactions, Total),
    Support is Count / Total.

count_containing(_, [], 0).
count_containing(Itemset, [T | Ts], Count) :-
    subset(Itemset, T), !,
    count_containing(Itemset, Ts, Count1),
    Count is Count1 + 1.
count_containing(Itemset, [_ | Ts], Count) :-
    count_containing(Itemset, Ts, Count).

% Confidenza di una regola: Itemset → Item
confidence(Itemset, Item, Conf) :-
    append(Itemset, [Item], Union),
    support(Union, SupUnion),
    support(Itemset, SupItemset),
    Conf is SupUnion / SupItemset.

% Trova regole con supporto e confidenza minimi
find_rules(MinSup, MinConf, Rule) :-
    transaction(T),
    subset(Itemset, T),
    item(Item),
    not(member(Item, Itemset)),
    support(Itemset, Sup), Sup >= MinSup,
    confidence(Itemset, Item, Conf), Conf >= MinConf,
    Rule = rule(Itemset, Item, Sup, Conf).
```

Clustering (K-Means)

```
% ===== K-MEANS SEMPLIFICATO =====

% Distanza euclidea
euclidean_dist(Point1, Point2, Dist) :-
    maplist_diff_sq(Point1, Point2, Squares),
    sum_list(Squares, Sum),
    Dist is sqrt(Sum).

maplist_diff_sq([], [], []).
maplist_diff_sq([H1 | T1], [H2 | T2], [Sq | Sqs]) :-
    Diff is H1 - H2,
    Sq is Diff * Diff,
    maplist_diff_sq(T1, T2, Sqs).

% Assegna punto al cluster più vicino
assign_cluster(Point, Centroids, Cluster) :-
```

```
    findall(D-C, (nth1(C, Centroids, Cent), euclidean_dist(Point, Cent, D)), Distances),
    keysort(Distances, [Cluster | _]).

% K-Means: iterazione
kmeans(Points, Centroids, Clusters) :-
    % Assegna ogni punto al cluster più vicino
    findall(C, (member(P, Points), assign_cluster(P, Centroids, C)), Assignments),
    % Raggruppa per cluster
    group_by_cluster(Assignments, Points, Clusters),
    % Ricalcola centroidi
    new_centroids(Clusters, NewCentroids),
    % Verifica convergenza
    ( NewCentroids = Centroids -> true
    ; kmeans(Points, NewCentroids, Clusters) ).
```

Learning da Esempi (Induzione)

```
% ===== INDUZIONE DI REGOLE =====
% Dati esempi positivi e negativi, induci regole generali

% Esempi: quali animali volano?
flies(eagle).
flies(sparrow).
flies(albatross).
flies(penguin).     % eccezione!

% Non volano:
¬flies(dog).
¬flies(cat).
¬flies(whale).

% Ipotesi: gli uccelli volano
bird(X) :- has_feathers(X).
bird(X) :- lays_eggs(X).

flies(X) :- bird(X), not(abnormal(X)).
abnormal(X) :- penguin(X).
abnormal(X) :- ostrich(X).
```

PARTE II - PARADIGMA FUNZIONALE (PYTHON)

Parte II - Python e Programmazione Funzionale

> **Docente:** Lorenzo Ferrone
> **Materiale:** Dispense del corso (146 pagine)

Introduzione a Python

Python è un linguaggio **multi-paradigma** che supporta:

- Programmazione funzionale (ma non puramente)
- Programmazione procedurale
- Programmazione ad oggetti

Filosofia Python:

- *Readability counts* - leggibilità prima di tutto
- *Batteries included* - libreria standard molto ricca
- Indentazione per definire i blocchi (no `{ }`)

```
# Java: 83 caratteri per "Hello World"
# Python: 8 caratteri
print("hello, world!")
```

Tipi di Dati in Python

Numerici

```
a = 1              # intero
pi = 3.14159       # float
z = 7 + 4j         # complesso
real = z.real      # parte reale
imag = z.imag      # parte immaginaria
```

Booleani

```
t = True
f = False

# Operazioni logiche
t and f    # False
t or f     # True
not t      # False
```

Stringhe

```
s = "questa e una stringa"
# Supporta Unicode nativamente

carattere = "a"    # e una stringa di 1 elemento (non c'e tipo char)

# Operazioni sulle stringhe
s = "hello world"
len(s)              # 11
s[0]                # 'h'
s[-1]               # 'd' (ultimo)
s[3:6]              # 'lo ' (slice: dal 3 incluso al 6 escluso)
"w" in s            # True
s.upper()           # "HELLO WORLD"
s.lower()           # "hello world"
s.index("l")        # 2 (prima occorrenza)

# Le stringhe sono IMMUTABILI!
s[0] = "H"          # TypeError!
```

Liste (MUTABILI)

```
L = [0, 1, 2, 3, 4, 5, 6, 7, 8]
vuota = []
```

```python
mista = [0, "uno", 2.0001, [3, 4]]   # tipi diversi!

# Accesso
L[0]        # 0
L[-1]       # 8 (ultimo)
L[3:6]      # [3, 4, 5] (slice)

# Modifica
L.append(9)         # aggiunge in fondo
L.extend([10, 11])  # aggiunge lista
L.insert(0, -1)     # inserisce alla posizione
L.remove(5)         # rimuove primo 5
L.pop()             # rimuove e ritorna ultimo
L.sort()            # ordina in place
L.reverse()         # inverte in place

# Ricerca
0 in L              # True
100 not in L        # True
L.index(3)          # posizione del 3
len(L)              # lunghezza

# List comprehension (FONDAMENTALE!)
quadrati = [i**2 for i in range(50)]           # [0, 1, 4, 9, 16, ...]
quad_pari = [i**2 for i in range(50) if i % 2 == 0]  # solo pari

# Esempio pratico: estrai vocali da stringa
S = "ciao mondo"
vocali = [c for c in S if c in "aeiou"]   # ['i', 'a', 'o', 'o', 'o']
```

Tuple (IMMUTABILI)

```python
T = (1, 2, 3)
# Come le liste ma NON si possono modificare!
T[0] = 5   # TypeError!

# Sono hashabili → possono essere chiavi di dizionario
```

Dizionari (Hash Map)

```python
D = {"uno": 1, "due": 2, "tre": 3}
vuoto = {}

# Accesso
D["uno"]            # 1
D.get("quattro", 0) # 0 (default se non esiste)

# Modifica
D["quattro"] = 4    # aggiunge/modifica
del D["uno"]        # cancella

# Iterazione
for chiave in D:
    print(chiave, D[chiave])

for chiave, valore in D.items():
    print(chiave, valore)

# Dict comprehension
```

```python
quadrati = {i: i**2 for i in range(10)}
# {0: 0, 1: 1, 2: 4, 3: 9, ...}
```

Controllo di Flusso

Condizionali

```python
x = 5
if x < 10:
    print("minore")
elif x == 10:
    print("uguale")
else:
    print("maggiore")

# Confronto multiplo (catena)
if 5 < x < 10:          # Pythonico!
    print("compreso")

# Ternary operator
risultato = "positivo" if x > 0 else "negativo"
```

Ciclo for

```python
# Itera su QUALUNQUE iterabile
stagioni = ["spring", "summer", "autumn", "winter"]
for s in stagioni:
    print(s)

# range(): ciclo numerico
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(3, 8):       # 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8 (passo 2)
    print(i)

# enumerate(): indice + valore
for i, s in enumerate(stagioni):
    print(f"{i}: {s}")

# zip(): itera su piu liste in parallelo
nomi = ["Anna", "Bob", "Cia"]
eta = [25, 30, 35]
for n, e in zip(nomi, eta):
    print(f"{n} ha {e} anni")
```

Ciclo while

```python
x = 0
while x < 10:
    print(x)
    x += 1

# break: esce dal ciclo
```

```python
# continue: salta all'iterazione successiva
for num in lista:
    if num < 0:
        break       # ferma al primo negativo
    if num == 0:
        continue    # salta gli zeri
    print(num)
```

Funzioni

Definizione di Base

```python
def add(a, b):
    return a + b

# Parametri opzionali (default)
def greet(name, greeting="Ciao"):
    return f"{greeting}, {name}!"

greet("Mario")           # "Ciao, Mario!"
greet("Mario", "Hello")  # "Hello, Mario!"
```

Args e Kwargs

```python
# *args: numero arbitrario di argomenti posizionali
def somma(*numeri):
    return sum(numeri)

somma(1, 2, 3, 4, 5)     # 15

# **kwargs: numero arbitrario di argomenti keyword
def info(**dati):
    for chiave, valore in dati.items():
        print(f"{chiave}: {valore}")

info(nome="Mario", eta=25, citta="Roma")

# Combinazione
def func(a, b, *args, **kwargs):
    pass
```

Funzioni come Oggetti di Prima Classe

```python
# Le funzioni sono oggetti come gli altri!

# In una lista
funzioni = [abs, int, round]
for f in funzioni:
    print(f(-3.7))   # 3.7, -3, -4

# Come parametro
def applica(f, x):
    return f(x)

applica(lambda x: x**2, 5)   # 25

# Come valore di ritorno (closure)
```

```python
def crea_moltiplicatore(n):
    def moltiplica(x):
        return x * n
    return moltiplica

doppio = crea_moltiplicatore(2)
triplo = crea_moltiplicatore(3)
doppio(5)    # 10
triplo(5)    # 15
```

Lambda-Calcolo (Teoria)

Il **lambda-calcolo** e il fondamento teorico della programmazione funzionale.

Sintassi

```
Termini:
  - Variabili: x, y, z
  - Astrazioni (funzioni): lambda x.M
  - Applicazioni: (M N)
```

Esempi

```
lambda x.x              funzione identita
lambda x.lambda y.x+y   funzione somma (curryficata)
(lambda x.x+5)(10)      applicazione -> 15
```

Beta-riduzione (applicazione di funzione)

```
(lambda x.M)(N)  ->  M[x := N]    (sostituisci x con N in M)

Esempio:
(lambda x. x + 5)(10)  ->  10 + 5  ->  15
(lambda x. lambda y. x + y)(2)(3)  ->  (lambda y. 2 + y)(3)  ->  2 + 3  ->  5
```

Currying

Le funzioni nel lambda-calcolo prendono un solo argomento. Per simulare funzioni a piu argomenti:

```python
# Currying in Python
def add(x):
    def fx(y):
        return y + x
    return fx

add(5)          # <function>
add(5)(6)       # 11

# Equivalente a:
add = lambda x: lambda y: x + y
add(5)(6)       # 11
```

Church Encoding (numeri come funzioni)

```python
# I numeri naturali come funzioni:
# n = lambda f: lambda x: f(f(...(f x))) n volte

def church_encode(n):
    from functools import reduce
    return lambda f: lambda x: reduce(lambda a, _: f(a), range(n), x)

# Zero
church_0 = lambda f: lambda x: x

# Successore
church_succ = lambda n: lambda f: lambda x: f(n(f)(x))

# Esempio: converti Church a intero
def church_to_int(c):
    return c(lambda x: x + 1)(0)

church_to_int(church_encode(3))   # 3
church_to_int(church_encode(5))   # 5
```

Programmazione Funzionale in Python

Funzioni Pure

```python
# Funzione PURA: stesso input -> stesso output, nessun effetto collaterale
def pure_add(x, y):
    return x + y

# Funzione IMPURA: ha effetti collaterali
total = 0
def impure_add(x):
    global total
    total += x   # modifica stato esterno!
    return total
```

Map, Filter, Reduce

```python
from functools import reduce
import operator

# ===== MAP =====
list(map(lambda x: x**2, [1, 2, 3, 4]))      # [1, 4, 9, 16]

# ===== FILTER =====
list(filter(lambda x: x > 0, [-2, -1, 0, 1, 2]))   # [1, 2]

# ===== REDUCE =====
reduce(lambda x, y: x + y, [1, 2, 3, 4, 5])   # 15
reduce(operator.mul, [1, 2, 3, 4, 5])           # 120 (5!)

# Fattoriale elegante
def fattoriale(n):
    return reduce(operator.mul, range(1, n + 1), 1)

fattoriale(5)   # 120
```

Lambda (Funzioni Anonime)

```python
# Sintassi: lambda parametri: espressione
quadrato = lambda x: x**2
quadrato(5)     # 25

somma = lambda x, y: x + y
somma(3, 4)     # 7

# Ordinamento personalizzato
studenti = [("Mario", "Rossi", 25), ("Luca", "Bianchi", 30), ("Anna", "Verdi", 22)]
sorted(studenti, key=lambda x: x[1])  # per cognome
sorted(studenti, key=lambda x: x[2], reverse=True)  # per eta decrescente
```

Iteratori e Generatori

```python
# ===== GENERATORI con yield =====
def fibonacci_infinito(a=0, b=1):
    while True:
        yield a
        a, b = b, a + b

import itertools
primi_10 = list(itertools.islice(fibonacci_infinito(), 10))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

sotto_100 = list(itertools.takewhile(lambda x: x < 100, fibonacci_infinito()))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]

# Generator expression (pigro)
quadrati_gen = (x**2 for x in range(1000000))
sum(x**2 for x in range(1000000))   # efficiente
```

itertools

```python
import itertools

itertools.cycle([1, 2, 3])           # 1,2,3,1,2,3,...
itertools.count(10, 2)               # 10,12,14,16,...
itertools.combinations([1,2,3,4], 2) # combinazioni
itertools.permutations([1,2,3])      # permutazioni
itertools.product(A, B)              # prodotto cartesiano
itertools.islice(iter, 5, 10)        # slice su iteratore
itertools.takewhile(cond, iter)      # prendi finche condizione vera
itertools.dropwhile(cond, iter)      # scarta finche condizione vera
```

Decoratori

```python
import time
from functools import lru_cache

# Cronometra esecuzione
def time_it(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.time()-start:.4f}s")
        return result
    return wrapper
```

```python
@time_it
def calcolo(n):
    return sum(i**2 for i in range(n))

# Memoizzazione
@lru_cache(maxsize=None)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)
fib(100)  # istantaneo!
```

Esercizi Pratici Python

Fibonacci

```python
def fibonacci(n, a=0, b=1):
    if n <= 0: return []
    sequenza = [a, b]
    for _ in range(2, n):
        a, b = b, a + b
        sequenza.append(a)
    return sequenza[:n]

# Versione generatore
def fib_gen(a=0, b=1):
    while True:
        yield a
        a, b = b, a + b
```

Numeri Primi

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

def sieve_eratosthenes(n):
    sieve = [True] * n
    sieve[0] = sieve[1] = False
    for i in range(2, int(n**0.5)+1):
        if sieve[i]:
            sieve[i*i:n:i] = [False]*len(sieve[i*i:n:i])
    return [i for i, p in enumerate(sieve) if p]
```

Composizione di Funzioni

```python
from functools import reduce

def compose(*functions):
    return reduce(lambda f, g: lambda x: f(g(x)), functions)

f = lambda x: 3*x
g = lambda x: x + 4
h = compose(f, g)   # h(x) = 3*(x+4)
h(2)                # 18
```

Accumulate

```
def accumulate(func, lista):
    if not lista: return []
    result = [lista[0]]
    for item in lista[1:]:
        result.append(func(result[-1], item))
    return result

accumulate(operator.add, [1,2,3,4,5])  # [1, 3, 6, 10, 15]
accumulate(operator.mul, [1,2,3,4,5])  # [1, 2, 6, 24, 120]
```

Esercizi d'Esame - Temi Tipici

Tema Prolog: Problemi Comuni

Percorso con somma massima in albero:

```
path_sum(leaf(V), [V], V).
path_sum(node(V, L, R), [V|Path], Sum) :-
    path_sum(L, Path, S), Sum is V + S.
path_sum(node(V, L, R), [V|Path], Sum) :-
    path_sum(R, Path, S), Sum is V + S.
```

Tutti i percorsi in un grafo:

```
all_paths(S, E, Path) :-
    path(S, E, [S], Rev), reverse(Rev, Path).
path(E, E, Vis, Vis).
path(C, E, Vis, Path) :-
    edge(C, N), not(member(N, Vis)), path(N, E, [N|Vis], Path).
```

Riassunto Completo

Prolog - Comandi Essenziali

Comando	Descrizione
`fact.`	Definisci un fatto
`rule :- body.`	Definisci una regola
`?- query.`	Esegui una query
`[H	T]`
`[]`	Lista vuota
`X is Expr`	Evalua espressione aritmetica
`X = Y`	Unifica X con Y
`X \= Y`	X diverso da Y
`!` (cut)	Blocca backtracking
`fail`	Fallimento esplicito
`findall(X, Goal, L)`	Trova tutte le soluzioni
`bagof(X, Goal, L)`	Trova tutte (raggruppate)

Comando	Descrizione
`setof(X, Goal, L)`	Trova tutte (ordinate, senza dup)
`assertz(F)`	Aggiungi fatto al database
`retract(F)`	Rimuovi fatto dal database
`listing(P)`	Mostra clausole di P
`var(X), nonvar(X)`	Controlla se variabile
`atom(X), number(X)`	Controlla tipo
`compound(X)`	E un termine composto
`ground(X)`	Non contiene variabili
`Term =.. List`	Decompone termine
`freeze(X, Goal)`	Goal ritardato finche X e istanziato
`dif(X, Y)`	X e Y devono rimanere diversi

Python - Funzioni Chiave per Programmazione Funzionale

Funzione	Descrizione
`map(f, iter)`	Applica f a ogni elemento
`filter(f, iter)`	Filtra elementi
`reduce(f, iter)`	Riduce a un valore
`lambda args: expr`	Funzione anonima
`[x for x in L]`	List comprehension
`(x for x in L)`	Generator expression
`yield x`	Generatore
`@decorator`	Decoratore
`*args, **kwargs`	Args arbitrari
`itertools.*`	Libreria iteratori
`functools.lru_cache`	Memoizzazione

> **Fine degli appunti completi del corso.**
>
> *"Readability counts. Special cases are not special enough to break the rules. Although practicality beats purity."*
> *- The Zen of Python*